

HANDS-ON ACTIVITY FOCUS GUIDE

Map and Verify the Motors

From Java names to real FTC hardware

25 MIN ACTIVITY	8 STUDENT ROLES	LIVE MOTOR TEST FORMAT	1 SAFETY LEAD
---------------------------	---------------------------	----------------------------------	-------------------------

ACTIVITY OUTCOME Students distinguish declaring a motor from mapping it, verify exact configuration names, and perform a controlled direction test without approaching an enabled drivetrain.

Materials

- Robot secured on stable blocks with both drive wheels fully clear.
- Robot Controller configuration and BasicTwoMotorTeleOp.java displayed side by side.
- Printed mapping and direction-observation sheet from this guide.
- Gamepad and Driver Station; POWERING ON safety poster visible at the test boundary.
- Optional printed mismatch cards: left_motor, leftMotor, Left_Motor, and left motor.

Before students enter

- Confirm the battery, wiring, wheel retention, and robot stand are secure.
- Verify the OpMode can initialize with the intended configuration before class.
- Choose the team's physical forward direction and mark it on the robot with tape.
- Set a boundary that nobody crosses while the OpMode is enabled.

SAFETY BOUNDARY Students never cross into the robot test area while the OpMode is enabled. STOP and visually confirm DISABLED before anyone approaches.

Student setup and roles

Assign one primary role to each student. Rotate roles on a second trial if time permits. The safety lead may stop the activity at any time.

STUDENT	ROLE	RESPONSIBILITY
1	Safety lead	Reads each POWERING ON step and watches the boundary.
2	Driver Station	Selects INIT, PLAY, and STOP only when directed.
3	Code reader	Finds declarations and hardwareMap.get() lines.
4	Config navigator	Reads the configured device names exactly.
5	Left observer	Records left-wheel direction from outside the boundary.
6	Right observer	Records right-wheel direction from outside the boundary.
7	Recorder	Completes the mapping and direction tables.
8	Debugger	Proposes one evidence-based correction after STOP.

Ground rules

- Predict before testing; record evidence before proposing a correction.
- Only the Driver Station operator touches INIT, PLAY, or STOP.
- Any student may call STOP for unexpected motion, instability, heat, noise, or an unclear boundary.
- Change only one variable between tests so the evidence remains interpretable.

POWERING ON gate

SAY AND DO ALL SIX STEPS 1. Secure the robot. 2. Confirm DISABLED. 3. Look around. 4. Make sure everyone is clear. 5. Call POWERING ON! and wait. 6. Visually confirm clear, then enable/PLAY.

After every test: press STOP, visually confirm DISABLED, and only then permit anyone to approach the robot.

25-minute teacher walkthrough

1. Separate declaration from connection | 4 minutes

TEACHER	<i>"Point to private DcMotor leftMotor;. Say: This creates a typed Java name. Has Java connected to the real motor yet?"</i>
---------	--

STUDENT	<i>"The code reader identifies DcMotor as the type and leftMotor as the variable. The class answers no connection exists yet."</i>
---------	--

ASK	<i>"What new job does hardwareMap.get() perform?"</i>
-----	---

LISTEN FOR It looks up a configured device and assigns that hardware object to the Java variable.

TEACHING NOTE Use a blank name tag as the analogy: the label exists before it is attached to a person or object.

2. Run an exact-name audit | 5 minutes

TEACHER	<i>"Read each hardwareMap.get() call from the inside out: device type, quoted configuration name, returned motor, assigned variable."</i>
---------	---

STUDENT	<i>"The config navigator reads the Robot Controller name aloud while the recorder compares every character, underscore, and capital."</i>
---------	---

ASK	<i>"Would leftMotor match left_motor?"</i>
-----	--

LISTEN FOR No. The Java variable may be named leftMotor, but the quoted configuration string must exactly match left_motor.

TEACHING NOTE Keep the Java variable name and configuration string in separate columns so students do not blend them together.

3. Predict a mismatch | 4 minutes

TEACHER	<i>"Show one incorrect name card. Ask students to predict when the failure occurs and whether PLAY should be pressed."</i>
---------	--

STUDENT	<i>"The debugger marks the mismatch, predicts an initialization error, and keeps the test at DISABLED."</i>
---------	---

ASK	<i>"Why is this not a motor-direction problem?"</i>
-----	---

LISTEN FOR	The program cannot find the device, so mapping fails before the driving loop can command it.
-------------------	--

TEACHING NOTE	A printed error challenge is enough. Do not intentionally alter a working robot configuration unless the teacher wants that extension.
----------------------	--

4. Perform the first direction test | 8 minutes

TEACHER	<i>"Robot secured. Confirm DISABLED. Look around. Make sure everyone is clear. Call POWERING ON! Wait. Visually confirm clear. Then direct the operator to INIT and PLAY."</i>
---------	--

STUDENT	<i>"The driver gently moves only the left stick, centers it, then only the right stick. Observers record which way the top of each wheel moves relative to the forward arrow."</i>
---------	--

ASK	<i>"What happens immediately if motion differs from the prediction?"</i>
-----	--

LISTEN FOR Press STOP, confirm DISABLED, and keep hands away until the disabled state is verified.

TEACHING NOTE Test one side at a time first. This isolates mapping and direction evidence.

5. Diagnose one variable at a time | 4 minutes

TEACHER	<i>"After STOP and DISABLED, compare commanded side, observed side, configuration name, and direction setting. Change only one cause before retesting."</i>
---------	---

STUDENT	<i>"The debugger states the evidence and proposes either a name correction or a direction-setting correction. The team repeats the complete safety sequence before another PLAY."</i>
---------	---

ASK	<i>"Is the right motor always the side that must be reversed?"</i>
-----	--

LISTEN FOR No. The required side depends on mounting, gearing, wiring, and the team's forward convention.

TEACHING NOTE Do not let students swap wires, touch wheels, or edit a live system while enabled.

Motor mapping and direction record

Student/team: _____ Date: _____ Robot: _____

CHECK	LEFT SIDE	RIGHT SIDE	EVIDENCE / NOTES
Java variable	leftMotor	rightMotor	
Device type	DcMotor.class	DcMotor.class	
Quoted code name			Copy exactly
Robot Configuration name			Copy exactly
Exact match?	Yes / No	Yes / No	Circle differences
Gentle forward-stick test	Wheel: ____	Wheel: ____	Expected: ____
Direction correction needed?	Yes / No	Yes / No	Only after DISABLED
STUDENT EVIDENCE RULE Write what was commanded and what was observed. Do not write a correction until STOP has been pressed and DISABLED has been confirmed.			

Safety sign-off

Before live testing, the safety lead confirms: robot secure ☐ DISABLED ☐ boundary clear ☐ POWERING ON called and wait completed ☐

After testing: STOP pressed ☐ DISABLED visually confirmed ☐ Safety lead initials: _____

COMPLETION CRITERIA The student uses lesson vocabulary, predicts before testing, records evidence, explains one result from the code or math, and follows STOP/DISABLED rules before approaching the robot.

Debrief

ASK	LISTEN FOR
What is created by <code>private DcMotor leftMotor;?</code>	A field/reference that can hold a DcMotor object; it is not yet mapped.
What are the two key <code>hardwareMap.get()</code> arguments?	The device type and the exact Robot Configuration name string.
Why do we test one side at a time?	It isolates which command is connected to which wheel and makes direction evidence easier to interpret.
What must happen before anyone approaches after a test?	STOP must be pressed and DISABLED must be visually confirmed.

Troubleshooting without guessing

OBSERVATION	NEXT CHECK AFTER STOP/DISABLED
INIT reports device not found	Compare quoted names character by character with Robot Configuration.
Left stick moves right wheel	Check which hub port and configured name represent each physical motor.
Both respond but one wheel is physically backward	After STOP/DISABLED, review that motor's Direction setting.
A wheel does not move	STOP; check telemetry command, configuration, wiring, battery, and mechanical binding one at a time.

Technical notes for the teacher

- A declaration creates a typed reference; `hardwareMap.get()` retrieves the configured hardware object.
- Configuration strings are exact identifiers. Case, spaces, and underscores matter.
- Commented direction commands do nothing until the leading `//` is removed.
- Direction correction is installation-specific; do not teach that one universal side must be reversed.
- The first test should use gentle input with the wheels clear of the table.

EXIT TICKET Explain the difference between `leftMotor` and `"left_motor"` in one or two sentences.